

Instrucciones de despliegue — Asistente Ico (Icompras360)

Documento dirigido al equipo de sistemas encargado de subir el asistente a producción. Describe la arquitectura, los requisitos y los pasos de despliegue.

1. Descripción general

Ico es un asistente conversacional para el soporte de la plataforma Icompras360. Responde las consultas de las farmacias usando el contenido de los manuales de la plataforma, mediante una arquitectura RAG (generación aumentada por recuperación).

Componentes:

Componente	Función	Cómo corre
Interfaz de chat	Frontend donde la farmacia consulta	Laravel (integrado en Icompras360)
n8n	Orquestación de los flujos	Contenedor Docker
Qdrant	Base de datos vectorial (conocimiento)	Contenedor Docker
Embeddings	Vectorización del texto	HuggingFace (vía n8n)
DeepSeek	Modelo de lenguaje que redacta	API externa (vía n8n)
MySQL	Identificación de farmacia, versión y registro de interacciones	Base de datos de la plataforma

Flujo de una consulta: la farmacia escribe en la interfaz → un webhook de n8n recibe la consulta → se consulta MySQL para identificar la farmacia y su versión (light/full) → se construye el perfil del usuario → el AI Agent interpreta con DeepSeek y recupera de Qdrant el fragmento pertinente de los manuales, filtrado por versión → DeepSeek redacta la respuesta → se devuelve a la interfaz. Cada interacción se registra en la tabla ai_mensajes_log.

2. Requisitos previos

- Servidor con Docker y Docker Compose instalados.
- Acceso a la base de datos MySQL de Icompras360.
- Credenciales (se entregan por separado y de forma segura):
 - API key de DeepSeek.

- Acceso al modelo de embeddings de HuggingFace.
 - Usuario, host y nombre de la base de datos MySQL.
 - Los dos flujos de n8n en formato JSON (adjuntos):
flujo_carga_conocimiento.json y flujo_conversacion.json.
 - Los manuales de la plataforma (carpeta manuales) para cargar la base de conocimiento.
-

3. Despliegue de Qdrant

Levantar el contenedor de Qdrant. En el entorno de desarrollo se usó:

```
docker run -p 6333:6333 -p 6334:6334 -v qdrant_storage:/qdrant/storage:z  
qdrant/qdrant
```

Para producción se recomienda definirlo en un docker-compose.yml junto con n8n, conservando el volumen persistente (qdrant_storage) para que la base de conocimiento no se pierda al reiniciar. Los puertos 6333 (HTTP) y 6334 (gRPC) deben quedar accesibles para n8n, pero no expuestos públicamente sin control de acceso.

4. Despliegue de n8n

Levantar el contenedor de n8n con un volumen persistente para conservar los flujos y las credenciales. La configuración del contenedor usado en desarrollo se adjunta en configuracion_n8n.txt (salida de docker inspect).

Consideraciones para producción:

- Definir la variable de entorno de la URL pública de n8n (WEBHOOK_URL / N8N_HOST) con el dominio real del servidor, ya que de ella depende la dirección del webhook al que apunta la interfaz.
 - Habilitar autenticación de acceso al panel de n8n.
 - Conservar el volumen de datos de n8n para persistencia.
-

5. Importar los flujos

Una vez n8n esté corriendo:

1. Ingresar al panel de n8n.

2. Importar flujo_carga_conocimiento.json (flujo que descarga los manuales, los fragmenta, los vectoriza y los carga en Qdrant).
3. Importar flujo_conversacion.json (flujo principal: webhook, consulta a MySQL, AI Agent con DeepSeek, memoria, recuperación en Qdrant y respuesta).

6. Configurar las credenciales

Las credenciales no viajan dentro de los JSON por seguridad. Dentro de n8n, en producción, hay que reconfigurar:

- Credencial de DeepSeek (API key) en el nodo del modelo de lenguaje.
- Credencial de HuggingFace en el nodo de embeddings (debe ser el mismo modelo de embeddings usado en desarrollo, para que los vectores sean compatibles).
- Conexión a MySQL en el nodo de consulta SQL (host, base de datos, usuario y contraseña de producción).

7. Cargar la base de conocimiento en Qdrant

Se recarga el conocimiento en producción usando el propio flujo de carga (no se migran vectores del entorno de desarrollo, para garantizar consistencia):

1. Colocar los manuales (carpeta manuales) en el origen que consume el flujo de carga (por ejemplo, la ubicación de descarga configurada en el nodo correspondiente).
2. Ejecutar el flujo flujo_carga_conocimiento.json.
3. El flujo fragmenta el contenido (chunks de 1200 con solapamiento de 150), genera los embeddings con HuggingFace y almacena los vectores en Qdrant, con los metadatos de módulo y versión.
4. Verificar que la colección quedó creada consultando: GET `http://<host-qdrant>:6333/collections`

Alternativa (si se prefiere migrar la colección existente en lugar de recargar): crear un snapshot de la colección en desarrollo (POST `/collections/<coleccion>/snapshots`), descargarlo y restaurarlo en el Qdrant de producción. La recarga es la vía recomendada.

8. Conectar la interfaz (Laravel)

- La interfaz de chat debe apuntar a la URL del webhook de n8n en producción, no a localhost. Actualizar la dirección del endpoint en la configuración del frontend.
 - Verificar que la interfaz envíe la consulta al webhook mediante la petición POST esperada por el flujo de conversación.
-

9. Verificación posterior al despliegue

Una vez desplegado, comprobar de extremo a extremo:

1. Enviar una consulta de prueba desde la interfaz y confirmar que se recibe respuesta.
 2. Probar con una farmacia de versión light y otra de versión full, y verificar que cada una recibe solo la información de su versión (filtrado por versión).
 3. Hacer una consulta cuya información no esté en los manuales y confirmar que el asistente responde con un mensaje controlado (no inventa).
 4. Verificar que la interacción quedó registrada en la tabla ai_mensajes_log.
 5. Revisar el panel de estadísticas.
-

10. Mantenimiento

Para actualizar la base de conocimiento cuando cambien los manuales, basta con reemplazar los archivos y volver a ejecutar el flujo de carga; no requiere reconstruir el sistema. Se recomienda mantener una nomenclatura estandarizada de los metadatos de módulo y versión para preservar la precisión de las búsquedas.